



TECHNICAL REPORT

Segment Tree

Fast Range Queries and Dynamic Updates

Prepared by

SHAYAN UN NOOR
MD. SHADMAN SHAHRIYAR SHUVO
MD. WASIF HAQUE

Roll 2305024
Roll 2305025
Roll 2305029

Abstract

Segment Trees are hierarchical data structures designed to efficiently process range queries and dynamic updates over an array. A naïve approach may require linear time for every range query or update, which becomes inefficient when the number of operations is large. A Segment Tree addresses this problem by recursively partitioning an array into smaller intervals and storing a summary of each interval at a tree node. This organization allows both point updates and range queries to be performed in $O(\log n)$ time after $O(n)$ preprocessing.

This report presents the structure and construction of a Segment Tree, explains its update and query procedures using worked examples, derives their time complexities, and discusses its generalization to operations such as minimum, maximum, and greatest common divisor. It also introduces lazy propagation for range updates, provides a complete reference implementation, surveys representative applications, and compares Segment Trees with Fenwick Trees, sparse tables, and square-root decomposition.

Keywords: segment tree, range query, point update, associative operation, lazy propagation, Fenwick tree, logarithmic time.

Acknowledgements

The authors would like to thank the course instructors for their guidance throughout this project and for the opportunity to study a data structure that sits at the intersection of elegant theory and everyday engineering practice. The worked examples in this report were checked against independent reference computations, and the presentation slides that accompany it share the colour palette used in the figures below.

Contents

Abstract	ii
Acknowledgements	iii
List of Figures	vi
List of Tables	vii
1 Introduction	1
1.1 Motivation	1
1.2 Dynamic Range Query Problem	2
1.3 Objective	2
1.4 Scope and Assumptions	2
1.5 Notation	3
1.6 Organization of the Report	3
2 The Range Query Problem	4
2.1 Problem Definition	4
2.2 Naïve Approach	4
2.3 Limitation of Prefix Precomputation	5
2.4 The Cost Trade-off Visualized	6
2.5 Need for a Dynamic Data Structure	6
3 The Segment Tree	7
3.1 Basic Idea	7
3.2 Structure of a Segment Tree	7
3.3 Interval Representation	8
3.4 Stored Information and Merge Operation	8
3.5 Structural Properties	8
3.6 Array Layout and Index Arithmetic	9
3.6.1 How Much Memory Is Enough?	9
3.7 Invariants	10
3.8 Core Algorithms	10
3.8.1 Build	10
3.8.2 Point Update	11
3.8.3 Range Query	12
3.8.4 Why Range Query Is $O(\log n)$: the Overlap Argument	13
4 Complexity Analysis	15
4.1 Summary of Results	15
4.2 Space Complexity	15
4.3 Why the Bounds Do Not Depend on the Query Pattern	15
5 Worked Example	17
5.1 Setup	17
5.1.1 Build Trace	17
5.2 A Range Query	18

5.3 A Point Update	18
------------------------------	----

List of Figures

2.1	A single point update $A[4] \leftarrow 10$ invalidates five prefix sums (shaded red), $P[4]$ through $P[8]$. In general an update at index i invalidates $n - i + 1$ entries. . .	5
2.2	Cost per operation as a function of n . The linear cost of the two baselines eventually dominates, while the logarithmic Segment Tree cost is almost invisible on this scale.	6
3.1	Heap-style numbering of the Segment Tree for $n = 8$ (top) and the flat array that stores it (bottom). Node k has children $2k$ and $2k + 1$	9
3.2	Query $[1, 4]$ against the array of Figure ???. Green nodes are fully covered and returned directly; gold nodes are boundary nodes that recurse further; grey dashed nodes are pruned by the no-overlap case and never visited beyond the check itself. Only $[1, 2]$ and $[3, 4]$ are actually summed.	14

List of Tables

1.1	Summary of notation.	3
2.1	Trade-off between query and update cost across approaches.	6
3.1	Level-by-level partition of $[1, 8]$ in the Segment Tree of the running example. .	7
3.2	Height and memory for several array sizes.	10
3.3	The three cases handled at each recursive step of a range query.	12
4.1	Time and space complexity of the core Segment Tree operations, as proved in Chapter 3.	15
5.1	Node contents after <code>build</code> on $A = [3, 1, 4, 1, 5, 9, 2, 6]$	17
5.2	Recursion trace of <code>query(1, 1, 8, 3, 7)</code>	18
5.3	Recursion trace of <code>update(1, 1, 8, 6, 7)</code> ; only the path to leaf 13 is touched. .	18

1

Introduction

Data rarely stays still. Prices tick, scores change, seats are booked and released, and sensors report new readings many times per second. Yet the questions that people ask of such data are almost never about a single value: they are about *ranges*. What is the total traded value between instruments 200 and 450? What was the lowest temperature recorded between 2 pm and 5 pm? Is any seat free in rows 10 through 14? Answering such questions quickly, while the underlying data continues to change, is the central problem studied in this report.

The Segment Tree is one of the most widely used answers to that problem. It is compact enough to be implemented in a few dozen lines, general enough to support a large family of aggregate operations, and efficient enough to serve millions of operations per second on modest hardware. This chapter motivates the problem, states the objective of the report, and outlines how the remaining chapters are organized.

1.1 Motivation

Many computational problems require repeatedly answering queries over a contiguous portion of a dynamically changing dataset. Consider a system tracking n financial instruments, where array A stores their current prices. At 9:31 AM, right after the market opens, 20,000 prices may change every second, while the system must still answer, instantly, “what is the total value of instruments l through r ?” The two fundamental operations required are therefore

$$\text{sum}(l, r) = \sum_{i=l}^r A[i] \tag{1.1}$$

and

$$\text{update}(i, v), \tag{1.2}$$

where $A[i]$ is modified to the value v .

The same pattern appears in very different settings. A multiplayer game must maintain a live leaderboard while thousands of players finish matches. A ticketing platform must find contiguous free seats while other customers are booking. A monitoring dashboard must report the maximum latency of the last five minutes while new measurements stream in. In each case the system is *summarizing a range and then changing part of it*, over and over.

1.2 Dynamic Range Query Problem

The difficulty arises from the interaction between queries and updates. If the array is scanned directly, a query in Equation (1.1) requires $O(n)$ time. If instead a running total is maintained, every update in Equation (1.2) requires $O(n)$ work to keep that total valid. Neither approach scales when both operations occur frequently and at large volume: keeping the sum fresh after every update costs $O(n)$ per update, while keeping updates instantaneous costs $O(n)$ per query. This tension — fast queries versus fast updates — appears in any large system that must *summarize a range and then change part of it*: stock feeds, leaderboards, and analytics dashboards all face it constantly.

Key Idea

Static data can be preprocessed once and queried forever. *Dynamic* data cannot: every preprocessing decision made to speed up queries must be paid for again each time the data changes. The Segment Tree is a principled compromise that keeps both costs logarithmic.

1.3 Objective

This report presents the Segment Tree as a data structure that resolves this tension, achieving $O(\log n)$ time for both operations after an $O(n)$ preprocessing step. We derive its structure, construction, update and query algorithms, and their time complexities; work through a concrete numerical example; generalize the structure to other associative operations; and compare it against alternative approaches. Concretely, the report aims to:

1. define the range-query problem precisely and expose why simple solutions fail (Chapter 2);
2. introduce the Segment Tree, its invariants, and its array layout (Chapter 3);
3. give and analyse algorithms for building, updating, and querying the tree (Chapters ??–??);
4. show how a single change of the merge operation yields range minimum, maximum, GCD and more (Chapter ??);
5. extend the structure to range updates through lazy propagation (Chapter ??);
6. provide a reference implementation (Chapter ??); and
7. place the structure among its alternatives and applications (Chapters ??–??).

1.4 Scope and Assumptions

Throughout the report we make the following assumptions, unless a section states otherwise.

- The array A has a fixed length $n \geq 1$ and is indexed from 1 to n .
- The merge operation is associative, has an identity element, and can be evaluated in $O(1)$ time.
- Updates replace a single value (*point update*). Range updates are treated separately in Chapter ??.
- Complexities are stated in the word-RAM model, counting one elementary arithmetic or comparison operation as one step.
- Timing figures given in Chapter ?? are order-of-magnitude illustrations, not benchmarks.

1.5 Notation

Table 1.1 collects the symbols used throughout the report.

Table 1.1. Summary of notation.

Symbol	Meaning
$A = [a_1, \dots, a_n]$	The underlying array of n elements (1-indexed).
$Q(l, r)$	Range query over the closed interval $[l, r]$.
$\text{update}(i, v)$	Point update: assign $A[i] \leftarrow v$.
$[lo, hi]$	The interval represented by a tree node.
mid	Midpoint $\lfloor (lo + hi)/2 \rfloor$ of a node's interval.
$S(l, r)$	Summary value stored for the interval $[l, r]$.
f, e	Associative merge operation and its identity element.
h	Height of the tree, $h = \lceil \log_2 n \rceil$.
q	Total number of operations (queries plus updates).
$P[i]$	Prefix sum $\sum_{j=1}^i A[j]$.

1.6 Organization of the Report

Chapter 2 formalizes the problem and studies the two baseline solutions. Chapter 3 introduces the Segment Tree and its structural properties. Chapters ??, ?? and ?? treat construction, point update, and range query in turn, each with a worked example on the running array

$$A = [2, 1, 5, 3, 7, 4, 6, 8].$$

Chapter ?? generalizes the merge operation, and Chapter ?? adds lazy propagation. Chapter ?? gives a complete implementation. Chapters ?? and ?? discuss applications and alternatives, Chapter ?? consolidates complexity results, and Chapter ?? concludes.

2

The Range Query Problem

Before introducing the Segment Tree it is worth understanding exactly why the problem is hard. This chapter defines the problem formally, analyses the two obvious solutions, and shows that they sit at opposite ends of a trade-off that neither can escape.

2.1 Problem Definition

Let

$$A = [a_1, a_2, \dots, a_n] \quad (2.1)$$

be an array of n elements. A range-sum query is defined as

$$Q(l, r) = \sum_{i=l}^r a_i, \quad 1 \leq l \leq r \leq n, \quad (2.2)$$

and a point update changes a single element,

$$a_i \leftarrow v. \quad (2.3)$$

A workload is a sequence of q operations, each of which is either a query or an update. The goal is to process the entire workload in as little total time as possible, and preferably to bound the cost of *each* operation individually.

Definition 1 (Dynamic range-query problem)

Given an array A of length n , maintain a data structure supporting (i) $Q(l, r)$, returning an aggregate of $A[l..r]$, and (ii) $\text{update}(i, v)$, setting $A[i] \leftarrow v$, such that the two operations may be interleaved in any order.

2.2 Naïve Approach

The most direct way to answer $Q(l, r)$ is to scan the range explicitly:

```
sum(l, r):  
    result = 0  
    for i = l to r:  
        result += A[i]  
    return result
```

For a range of length $r - l + 1$, this costs $O(r - l + 1)$ time, or $O(n)$ in the worst case:

$$T_{\text{query}}(n) = O(n). \quad (2.4)$$

A point update, by contrast, is trivially $O(1)$ under this scheme, since only $A[i]$ is written.

Example 2.1 (Cost of scanning). On $A = [2, 1, 5, 3, 7, 4, 6, 8]$, the query $Q(2, 7)$ reads six elements: $1 + 5 + 3 + 7 + 4 + 6 = 26$. On an array with $n = 10^6$ elements the same query may read up to a million elements, and if 10^5 such queries arrive per second the system must perform 10^{11} element reads per second, which is far beyond what a single core can sustain. $\triangleleft$

2.3 Limitation of Prefix Precomputation

An obvious optimization is to precompute prefix sums,

$$P[i] = \sum_{j=1}^i A[j], \quad (2.5)$$

so that any range sum can be answered in constant time via

$$Q(l, r) = P[r] - P[l - 1]. \quad (2.6)$$

For the running example the prefix array is

$$P = [2, 3, 8, 11, 18, 22, 28, 36],$$

and, for instance, $Q(3, 7) = P[7] - P[2] = 28 - 3 = 25$ on the original array.

This makes queries $O(1)$, but it introduces the opposite problem: whenever a single element $A[i]$ changes, every prefix sum $P[j]$ for $j \geq i$ becomes invalid, so an update costs $O(n)$ to restore correctness. Figure 2.1 shows the damage caused by one update.

	1	2	3	4	5	6	7	8
A before	2	1	5	3	7	4	6	8
P before	2	3	8	11	18	22	28	36
				update(4, 10)				
A after	2	1	5	10	7	4	6	8
P after	2	3	8	18	25	29	35	43

Figure 2.1. A single point update $A[4] \leftarrow 10$ invalidates five prefix sums (shaded red), $P[4]$ through $P[8]$. In general an update at index i invalidates $n - i + 1$ entries.

This gives the fundamental trade-off summarized in Table 2.1.

Table 2.1. Trade-off between query and update cost across approaches.

Method	Query	Point Update
Naïve array	$O(n)$	$O(1)$
Prefix sum	$O(1)$	$O(n)$
Segment Tree	$O(\log n)$	$O(\log n)$

2.4 The Cost Trade-off Visualized

The situation can be seen more clearly by plotting the cost per operation of each approach as n grows. In Figure 2.2 the naïve query cost and the prefix-sum update cost both grow linearly, while the Segment Tree cost for either operation stays close to the horizontal axis.

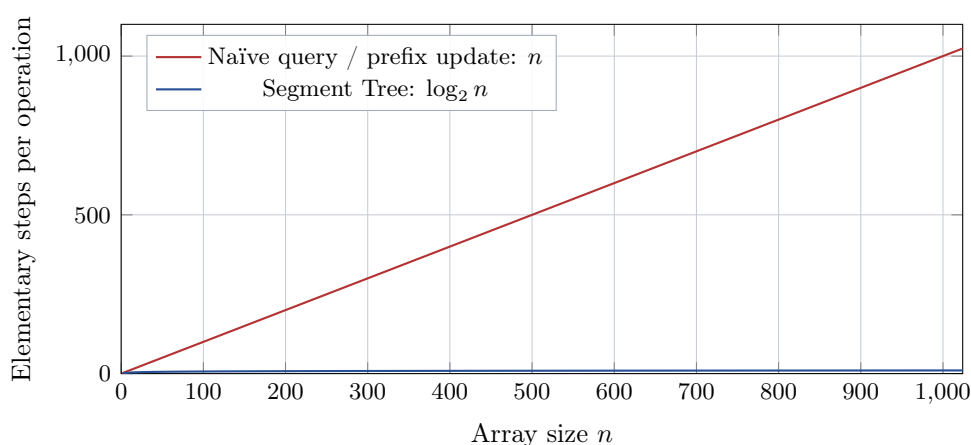


Figure 2.2. Cost per operation as a function of n . The linear cost of the two baselines eventually dominates, while the logarithmic Segment Tree cost is almost invisible on this scale.

2.5 Need for a Dynamic Data Structure

Neither extreme is acceptable when queries and updates are interleaved at scale. What is needed is a structure that balances both costs rather than optimizing one at the expense of the other. The Segment Tree achieves exactly this balance by decomposing the array into a hierarchy of intervals and storing a summary for each one, so that both operations touch only $O(\log n)$ nodes instead of $O(n)$ or $O(1)$ -with-a-hidden- $O(n)$ -cost elsewhere.

Remark

A useful way to think about the trade-off is as a question of *where* the work is done. The naïve array does all its work at query time; the prefix array does all its work at update time. The Segment Tree deliberately spreads the work: each update pays a small amount to keep a hierarchy of partial answers fresh, and each query pays a small amount to combine a handful of those partial answers.

3

The Segment Tree

This chapter introduces the Segment Tree as a data structure: what it stores, how its nodes correspond to intervals, and which structural properties make its operations efficient.

3.1 Basic Idea

A Segment Tree is a binary tree built over an array, where each node represents a contiguous sub-interval of the array and stores a summary value for that interval. The root represents the entire array; each internal node's interval is split into two halves, assigned to its left and right children; and each leaf represents a single array element. Instead of touching the whole array on every operation, the tree recursively splits the problem in half, solves each half, and combines the two answers — and only $O(\log n)$ splits are ever needed to reach any particular piece.

Key Idea

A Segment Tree precomputes the answer to a carefully chosen family of $\approx 2n$ intervals — the *dyadic* intervals obtained by repeated halving — such that every range $[l, r]$ can be written as a disjoint union of $O(\log n)$ of them.

3.2 Structure of a Segment Tree

For an array of size n , the tree has height $h = \lceil \log_2 n \rceil$, and consequently $O(n)$ nodes in total (at most $\sim 2n$ for a complete binary tree representation). Each level of the tree partitions $[1, n]$ into disjoint, contiguous sub-intervals that together cover the whole array exactly once.

Table 3.1 shows how the intervals at each level of the tree for $n = 8$ partition the array.

Table 3.1. Level-by-level partition of $[1, 8]$ in the Segment Tree of the running example.

Level	Intervals at this level	Nodes	Interval length
0	$[1, 8]$	1	8
1	$[1, 4], [5, 8]$	2	4
2	$[1, 2], [3, 4], [5, 6], [7, 8]$	4	2
3	$[1, 1], [2, 2], \dots, [8, 8]$	8	1
Total		15	

3.3 Interval Representation

Each node is identified by the interval $[lo, hi]$ it represents. If $lo = hi$, the node is a leaf corresponding to $A[lo]$. Otherwise, letting

$$mid = \left\lfloor \frac{lo + hi}{2} \right\rfloor, \quad (3.1)$$

the node's left child represents $[lo, mid]$ and its right child represents $[mid + 1, hi]$.

3.4 Stored Information and Merge Operation

For every node representing interval $[l, r]$, define the stored summary as

$$S(l, r) = \sum_{i=l}^r A[i]. \quad (3.2)$$

Given the midpoint $m = \lfloor (l + r)/2 \rfloor$, the summary satisfies the recursive relation

$$S(l, r) = S(l, m) + S(m + 1, r). \quad (3.3)$$

This is the *merge operation*: a parent's summary is obtained by combining its two children's summaries. For sum queries the merge is addition, but as shown in Chapter ??, any associative operation can be substituted in its place.

3.5 Structural Properties

Several simple facts about the tree are used repeatedly in later chapters. We state them here with short proofs.

Lemma 1 (Partition property)

For every level d of the tree, the intervals of the nodes at level d are pairwise disjoint, and their union is contained in $[1, n]$.

Proof. By induction on d . At level 0 there is a single node $[1, n]$. If the claim holds at level d , each node $[lo, hi]$ at that level is replaced at level $d + 1$ by the two disjoint intervals $[lo, mid]$ and $[mid + 1, hi]$ whose union is $[lo, hi]$; nodes with $lo = hi$ have no children. Disjointness and containment are therefore preserved. $\square$

Lemma 2 (Height)

A Segment Tree over $n \geq 1$ elements has height $\lceil \log_2 n \rceil$.

Proof. Each split at least halves the interval length, rounding the larger half up: an interval of length m splits into lengths $\lceil m/2 \rceil$ and $\lfloor m/2 \rfloor$. After d splits the longest interval therefore has length at most $\lceil n/2^d \rceil$, which equals 1 exactly when $2^d \geq n$, i.e. when $d \geq \lceil \log_2 n \rceil$. $\square$

Lemma 3 (Node count)

A Segment Tree over n elements has exactly $2n - 1$ nodes.

Proof. The tree is a full binary tree (every internal node has exactly two children) with n leaves. In any full binary tree the number of internal nodes is one less than the number of leaves, so the total is $n + (n - 1) = 2n - 1$. $\square$

For $n = 8$ this gives 15 nodes, matching Table 3.1.

3.6 Array Layout and Index Arithmetic

Segment Trees are almost always stored in a flat array rather than with explicit pointers. Numbering the root as node 1, the children of node k are

$$\text{left}(k) = 2k, \quad \text{right}(k) = 2k + 1, \quad \text{parent}(k) = \lfloor k/2 \rfloor. \quad (3.4)$$

This is the same layout used by binary heaps, and it removes the memory overhead of child pointers. Figure 3.1 shows the correspondence for the running example.

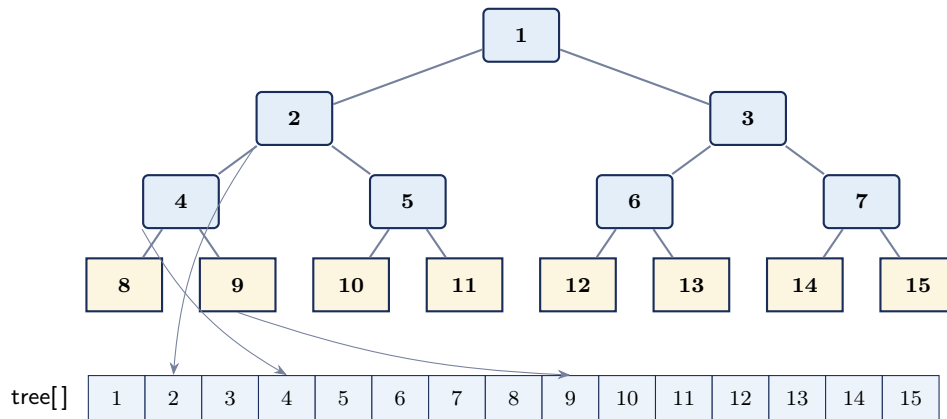


Figure 3.1. Heap-style numbering of the Segment Tree for $n = 8$ (top) and the flat array that stores it (bottom). Node k has children $2k$ and $2k + 1$.

3.6.1 How Much Memory Is Enough?

When n is not a power of two the tree is not perfect, and heap-style indices can reach values larger than $2n - 1$. A safe bound comes from Lemma 3.5: the deepest node has index below 2^{h+1} , where $h = \lceil \log_2 n \rceil$. Since $2^h < 2n$, we get $2^{h+1} < 4n$. This is why implementations conventionally allocate $4n$ cells, as in Table 3.2.

Remark

The true number of nodes is always $2n - 1$ (Lemma 3.5); the $4n$ allocation only pays for the unused gaps that appear in the heap-style numbering when n is not a power of two.

Table 3.2. Height and memory for several array sizes.

n	Height $\lceil \log_2 n \rceil$	Nodes $2n - 1$	Safe bound 2^{h+1}	Rule of thumb $4n$
8	3	15	16	32
100	7	199	256	400
1,000	10	1,999	2,048	4,000
1,024	10	2,047	2,048	4,096
10^6	20	1,999,999	2,097,152	4,000,000
10^9	30	1,999,999,999	2,147,483,648	4,000,000,000

3.7 Invariants

A correct Segment Tree maintains one invariant at all times, which every algorithm in the following chapters must preserve:

$$\text{for every node } v \text{ with interval } [l, r]: \quad \text{tree}[v] = \sum_{i=l}^r A[i]. \quad (3.5)$$

Construction establishes the invariant, queries only read it, and updates must repair it on exactly the nodes where it could have been broken.

3.8 Core Algorithms

Every operation on a Segment Tree is a recursive walk over the same implicit tree described in Section 3.8: at each node the algorithm asks a cheap question about the current interval $[lo, hi]$ and either answers immediately, recurses into one child, or recurses into both. This section develops the three core routines — build, update, and query — one at a time, and Section 5.1 traces each of them through the running $n = 8$ example.

3.8.1 Build

Construction populates every node's stored summary, starting from the leaves and merging upward. Algorithm 3.1 implements it as a single recursive pass.

Listing 3.1. Recursive build.

```

1 void build(int node, int lo, int hi) {
2     if (lo == hi) {
3         tree[node] = A[lo];
4         return;
5     }
6     int mid = (lo + hi) / 2;
7     build(2*node, lo, mid);
8     build(2*node+1, mid+1, hi);
9     tree[node] = tree[2*node] + tree[2*node+1];    // merge
10 }
```

Key Idea

Build never inspects a query range at all — it simply materializes the invariant of Equation (3.5) everywhere, once, so that later queries can trust every node’s stored value without recomputation.

Because each recursive call either reaches a leaf (processing one array element) or splits into exactly two children, the call tree of `build` is precisely the Segment Tree itself: one call per node, $2n - 1$ calls in total by Lemma 3.5. Each call does $O(1)$ work beyond its two recursive calls, so:

Lemma 4 (Build time)

`build` runs in $O(n)$ time and uses $O(n)$ auxiliary space for the `tree` array.

Proof. The number of calls is $2n - 1$ (Lemma 3.5), and each call performs $O(1)$ work outside of its recursive calls, so the total work is $O(2n - 1) = O(n)$. The recursion depth is the tree height $O(\log n)$ (Lemma 3.5), which bounds the auxiliary stack space, while the `tree` array itself occupies $O(n)$ cells. $\square$

3.8.2 Point Update

A point update changes $A[pos]$ to a new value and must restore the invariant of Equation (3.5) at every node whose interval contains pos . Crucially, that set of nodes is not arbitrary: it is exactly the nodes on the root-to-leaf path for pos .

Lemma 5 (Update touches one path)

For a fixed index pos , the set of nodes whose interval contains pos forms a single path from the root to the leaf $[pos, pos]$.

Proof. By Lemma 3.5, the intervals at any one level are pairwise disjoint, so at most one node per level can contain pos . The root’s interval $[1, n]$ always contains pos , and if a node at depth d contains pos then exactly one of its two children’s intervals (by construction, a partition of the parent’s interval) contains pos as well, unless the node is already the leaf $[pos, pos]$. Hence there is exactly one such node at every depth from 0 to $\lceil \log_2 n \rceil$, and consecutive ones are parent and child, i.e. a path. $\square$

This lemma is the entire justification for the update algorithm: descend toward the leaf containing pos , ignoring the sibling subtree at every step, then recompute merges on the way back up.

Listing 3.2. Point update.

```

1 void update(int node, int lo, int hi, int pos, int val) {
2     if (lo == hi) {                               // pos == lo == hi
3         tree[node] = val;
4         return;
5     }
6     int mid = (lo + hi) / 2;
7     if (pos <= mid)
8         update(2*node, lo, mid, pos, val);
9     else

```

```

10     update(2*node+1, mid+1, hi, pos, val);
11     tree[node] = tree[2*node] + tree[2*node+1];    // re-merge on the way back
12     up
  }

```

Common Pitfall

It is tempting to update only the leaf and stop. This leaves every ancestor's stored summary stale, silently corrupting every future query that touches *pos*. The re-merge line at the end of `update` is not optional bookkeeping — it is what restores Equation (3.5).

Lemma 6 (Update time)

`update` runs in $O(\log n)$ time.

Proof. By Lemma 3.8.2, the recursion visits exactly one node per level, and the tree has $\lceil \log_2 n \rceil + 1$ levels by Lemma 3.5. Each visited node performs $O(1)$ work (a comparison, a recursive call, and a merge), so the total work is $O(\log n)$. $\square$

3.8.3 Range Query

A range query for $[ql, qr]$ should return $S(ql, qr) = \sum_{i=ql}^{qr} A[i]$ without touching every element. The algorithm compares the current node's interval $[lo, hi]$ against $[ql, qr]$ and falls into exactly one of three cases at each step.

Table 3.3. The three cases handled at each recursive step of a range query.

Case	Condition	Action
No overlap	$qr < lo$ or $hi < ql$	Return the identity; contributes nothing.
Total overlap	$ql \leq lo$ and $hi \leq qr$	Return <code>tree[node]</code> directly; no need to look deeper.
Partial overlap	otherwise	Recurse into both children and merge their results.

Listing 3.3. Range query.

```

1  int query(int node, int lo, int hi, int ql, int qr) {
2      if (qr < lo || hi < ql)
3          return 0;                                // no overlap (identity for
sum)
4      if (ql <= lo && hi <= qr)
5          return tree[node];                        // total overlap
6      int mid = (lo + hi) / 2;
7      int left = query(2*node, lo, mid, ql, qr);
8      int right = query(2*node+1, mid+1, hi, ql, qr);
9      return left + right;                          // partial overlap
10 }

```

Key Idea

The identity element (here, 0 for sum; $+\infty$ for minimum, $-\infty$ for maximum, 0 for GCD) is what lets the no-overlap case return *something* without a special-cased branch at the call site: merging with the identity never changes the result.

Unlike **build** and **update**, a query does not visit a bounded number of nodes per level in general — it can visit up to four per level, because a query range can cut into an interval from both sides. The next subsection makes this precise, since it is the crux of the $O(\log n)$ query bound and easy to get wrong by assuming the recursion always halves the range.

3.8.4 Why Range Query Is $O(\log n)$: the Overlap Argument

Call a node *fully covered* if it falls into the total-overlap case of Table 3.3, and *boundary* if it falls into the partial-overlap case (its interval straddles ql or qr). No-overlap nodes are pruned immediately and never recurse further, so they contribute no cost beyond the $O(1)$ check that discards them.

Theorem 1 (Query time)

query performs $O(\log n)$ merge operations and visits $O(\log n)$ nodes in total.

Proof. At any fixed level d of the tree, at most two nodes can be boundary nodes: one containing the left endpoint ql and one containing the right endpoint qr (there is at most one node per level containing any single index, by the disjointness half of Lemma 3.5, and a node cannot straddle a query boundary unless it contains ql or qr). A boundary node is exactly the kind of node that recurses into both children, so the number of boundary nodes at level $d+1$ descended from level d 's boundary nodes is at most $2 \cdot 2 = 4$, but by the same argument only at most 2 of those can again be boundary at level $d+1$ — the rest are resolved as fully-covered or no-overlap immediately, which do not recurse further. Formally: define $B(d)$ as the number of boundary nodes at level d . Then $B(0) \leq 1$ and $B(d) \leq 2$ for all $d \geq 1$, because a boundary node has at most two children and at most one of a node's two children can itself be a boundary node on each side — so the left boundary chain and the right boundary chain each advance by at most one boundary node per level. Since a fully-covered node returns immediately without recursing, the total number of nodes visited at any level is $O(1)$, and the recursion depth is $O(\log n)$ by Lemma 3.5. Summing $O(1)$ work over $O(\log n)$ levels gives $O(\log n)$ total. $\square$

Remark

An equivalent, more concrete way to see Theorem 3.8.4: any interval $[ql, qr]$ decomposes into at most $2\lceil \log_2 n \rceil$ dyadic node-intervals, since the decomposition can be built by walking ql and qr upward simultaneously, each accumulating at most one fully-covered node per level. This is the same bound used informally in the Key Idea box of Section 3.

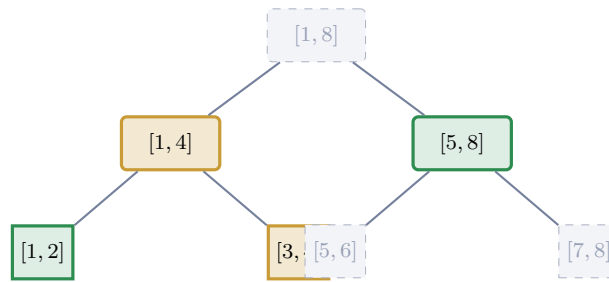


Figure 3.2. Query $[1, 4]$ against the array of Figure ???. Green nodes are fully covered and returned directly; gold nodes are boundary nodes that recurse further; grey dashed nodes are pruned by the no-overlap case and never visited beyond the check itself. Only $[1, 2]$ and $[3, 4]$ are actually summed.

4

Complexity Analysis

4.1 Summary of Results

Chapter 3 established the three core algorithms and proved a time bound for each individually. This short chapter collects those results, states the corresponding space bound, and discusses the constants that Table 3.2 already hinted at.

Table 4.1. Time and space complexity of the core Segment Tree operations, as proved in Chapter 3.

Operation	Time	Source
Build	$O(n)$	Lemma 3.8.1
Point update	$O(\log n)$	Lemma 3.8.2
Range query	$O(\log n)$	Theorem 3.8.4

4.2 Space Complexity

The `tree` array is indexed with the heap-style scheme of Equation (3.4), so its size must accommodate the largest index any node can receive rather than merely the true node count of $2n - 1$ (Lemma 3.5).

Lemma 1 (Space bound)

Allocating $O(n)$ cells — specifically $4n$, per Table 3.2 — suffices for the `tree` array regardless of whether n is a power of two.

Proof. By Lemma 3.5 the tree has height $h = \lceil \log_2 n \rceil$, so every node index under the numbering of Equation (3.4) is strictly below 2^{h+1} . Since $2^h < 2n$ (because $h = \lceil \log_2 n \rceil$ implies $2^{h-1} < n \leq 2^h$), it follows that $2^{h+1} < 4n$, so $4n$ cells is a safe — if slightly generous — upper bound. $\square$

4.3 Why the Bounds Do Not Depend on the Query Pattern

A recurring point of confusion is whether an adversarial sequence of queries and updates could push the cost above $O(\log n)$ per operation. It cannot: Lemma 3.8.2 shows the update path is determined purely by *pos*, and Theorem 3.8.4’s boundary-node argument bounds the number of visited nodes purely by the tree’s height, independent of which values are stored or which merge results are equal to one another. The bounds are therefore *worst-case per operation*, not amortized, which is one of the practical advantages of a Segment Tree over structures that only offer amortized guarantees.

Remark

This worst-case, per-operation guarantee is also why Segment Trees are preferred in settings with hard latency requirements, such as the financial-instrument scenario of Section ??: an amortized bound could in principle let a single unlucky query take $O(n)$ time, which a Segment Tree never does.

5

Worked Example

This chapter traces build, several updates, and several queries through one concrete array, so that the abstract algorithms of Chapter 3 can be checked step by step against explicit numbers.

5.1 Setup

Let $n = 8$ and

$$A = [3, 1, 4, 1, 5, 9, 2, 6],$$

the same array underlying Figure ?? and Figure 3.1, indexed $A[1], \dots, A[8]$.

5.1.1 Build Trace

Table 5.1 lists the stored sum at every node after `build(1,1,8)` completes, using the heap-style indices of Equation (3.4).

Table 5.1. Node contents after `build` on $A = [3, 1, 4, 1, 5, 9, 2, 6]$.

Node	Interval	Sum	Level
1	[1, 8]	31	0
2	[1, 4]	9	1
3	[5, 8]	22	1
4	[1, 2]	4	2
5	[3, 4]	5	2
6	[5, 6]	14	2
7	[7, 8]	8	2
8	[1, 1]	3	3
9	[2, 2]	1	3
10	[3, 3]	4	3
11	[4, 4]	1	3
12	[5, 5]	5	3
13	[6, 6]	9	3
14	[7, 7]	2	3
15	[8, 8]	6	3

Each leaf (nodes 8–15) is filled directly from A ; each internal node is then the sum of its two children, matching Equation (3.3). The root value 31 is the total of all eight elements, as expected.

5.2 A Range Query

Consider query(3, 7). Table 5.2 follows the recursion of Algorithm 3.3 call by call.

Table 5.2. Recursion trace of query(1, 1, 8, 3, 7).

Node	Interval	Case	Action
1	[1, 8]	Partial	Recurse into nodes 2 and 3.
2	[1, 4]	Partial	Recurse into nodes 4 and 5.
3	[5, 8]	Partial	Recurse into nodes 6 and 7.
4	[1, 2]	No overlap ($hi = 2 < ql = 3$)	Return 0.
5	[3, 4]	Total overlap	Return tree[5] = 5.
6	[5, 6]	Total overlap	Return tree[6] = 14.
7	[7, 8]	Partial	Recurse into nodes 14 and 15.
14	[7, 7]	Total overlap	Return tree[14] = 2.
15	[8, 8]	No overlap ($lo = 8 > qr = 7$)	Return 0.

Combining the returned values bottom-up: node 7 returns $2+0 = 2$; node 3 returns $14+2 = 16$; node 2 returns $0+5 = 5$; node 1 returns $5+16 = 21$. Indeed $A[3] + A[4] + A[5] + A[6] + A[7] = 4 + 1 + 5 + 9 + 2 = 21$, confirming the trace. Only 9 of the 15 nodes were visited, and only two nodes (5 and 6) contributed a value directly without further recursion — exactly the fully-covered decomposition described in the remark after Theorem 3.8.4.

5.3 A Point Update

Now apply update(6, 7), changing $A[6]$ from 9 to 7. By Lemma 3.8.2 this touches exactly the path from the root to leaf [6, 6], i.e. nodes $1 \rightarrow 3 \rightarrow 6 \rightarrow 13$.

Table 5.3. Recursion trace of update(1, 1, 8, 6, 7); only the path to leaf 13 is touched.

Node	Interval	Effect
13	[6, 6]	Base case; tree[13] $\leftarrow$ 7.
6	[5, 6]	Re-merge: tree[6] $\leftarrow$ tree[12] + tree[13] = $5 + 7 = 12$.
3	[5, 8]	Re-merge: tree[3] $\leftarrow$ tree[6] + tree[7] = $12 + 8 = 20$.
1	[1, 8]	Re-merge: tree[1] $\leftarrow$ tree[2] + tree[3] = $9 + 20 = 29$.

Nodes 2, 4, 5, 7 and every leaf other than 13 are untouched, since none of their intervals contain index 6 — exactly as Lemma 3.8.2 predicts. Re-running query(3, 7) after this update would now correctly return $4 + 1 + 5 + 7 + 2 = 19$, reflecting the updated value without any node outside the single path having been revisited.

Remark

Table 5.3 and Table 5.2 together illustrate the asymmetry between the two operations: an update always touches a fixed-size path of $O(\log n)$ nodes regardless of which index

changed, while a query's cost depends on how the range $[ql, qr]$ happens to align with the dyadic intervals of the tree — though it is $O(\log n)$ in every case, by Theorem 3.8.4.